

Anexo Módulo 4

(A) CÓDIGO VHDL PARA HPU (Holon Processing Unit)

Arquivo: hpu_pic_sft.vhdl

Versão: 1.0

Descrição: Implementação em VHDL do processador holônico para inferência ativa, com pipeline de 6 estágios, operação em FP16, e interface com HAM e HI.

```
``vhdl
-- =====
-- HPU (Holon Processing Unit) – PIC/SFT Native Processor
-- Versão: 1.0
-- Descrição: Pipeline de 6 estágios para inferência ativa:
--      Leitura → Erro → Saliência → Atualização  $\mu$ 
--      → Atualização precisão → Escrita
-- =====

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.fixed_pkg.all;
use work.fp16_pkg.all; -- Pacote com funções FP16 (multiplicação, soma, divisão)

entity HPU is
  generic (
    K : integer := 4;          -- Número de módulos (holons)
    DATA_WIDTH : integer := 16; -- FP16 (16 bits)
    ADDR_WIDTH : integer := 8;  -- Endereçamento da HAM (256 entradas)
    FRAC_BITS : integer := 10   -- Bits fracionários para fixed-point
  );
  port (
    clk : in std_logic;        -- Clock (1 GHz)
    rst : in std_logic;        -- Reset síncrono
    enable : in std_logic;      -- Habilitar processamento

    -- Interface com sensores (entrada obs)
    obs_in : in sfixed(DATA_WIDTH-1 downto -FRAC_BITS); -- Observação (K valores)

    -- Interface com HAM (memória associativa)
    ham_addr_out : out std_logic_vector(ADDR_WIDTH-1 downto 0);
    ham_data_in : in std_logic_vector(DATA_WIDTH-1 downto 0); --  $\mu$ , precision,
err_ema_sq
    ham_data_out : out std_logic_vector(DATA_WIDTH-1 downto 0);
```

```

    ham_we : out std_logic;      -- Write enable
    ham_ready : in std_logic;    -- HAM pronto

    -- Interface com HI (barramento)
    hi_tx : out std_logic;       -- Transmissão para HI
    hi_rx : in std_logic;        -- Recepção do HI (broadcast)
    hi_ready : out std_logic;    -- HPU pronto para enviar

    -- Controle de energia (modulação por  $\phi_S$ )
    phi_S_in : in sfixed(DATA_WIDTH-1 downto -FRAC_BITS); --  $\phi_S$  (0 a 1)
    energy_mode : out std_logic_vector(1 downto 0) -- 00: econômico, 01: normal, 10: alto
    desempenho
    );
end HPU;

```

architecture rtl of HPU is

```

    -- Pipeline stages registers
    type stage1_reg_type is record
        obs : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        mu : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        precision : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        err_ema_sq : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        addr : std_logic_vector(ADDR_WIDTH-1 downto 0);
    end record;
    signal s1_reg : stage1_reg_type;

    type stage2_reg_type is record
        err : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        mu : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        precision : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        err_ema_sq : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        addr : std_logic_vector(ADDR_WIDTH-1 downto 0);
    end record;
    signal s2_reg : stage2_reg_type;

    type stage3_reg_type is record
        salience : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        mu : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        precision : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        err_ema_sq : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
        addr : std_logic_vector(ADDR_WIDTH-1 downto 0);
    end record;
    signal s3_reg : stage3_reg_type;

```

```

type stage4_reg_type is record
    mu_new : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    precision : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    err_ema_sq : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    addr : std_logic_vector(ADDR_WIDTH-1 downto 0);
end record;
signal s4_reg : stage4_reg_type;

type stage5_reg_type is record
    mu_new : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    precision_new : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    err_ema_sq_new : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    addr : std_logic_vector(ADDR_WIDTH-1 downto 0);
end record;
signal s5_reg : stage5_reg_type;

-- Constantes do modelo (fixas, podem ser ajustadas via registradores)
constant KAPPA : sfixed(DATA_WIDTH-1 downto -FRAC_BITS) := to_sfised(0.25,
DATA_WIDTH-1, -FRAC_BITS);
constant PRECISION_EMA : sfixed(DATA_WIDTH-1 downto -FRAC_BITS) := to_sfised(0.05,
DATA_WIDTH-1, -FRAC_BITS);
constant PRECISION_MIN : sfixed(DATA_WIDTH-1 downto -FRAC_BITS) := to_sfised(0.1,
DATA_WIDTH-1, -FRAC_BITS);
constant PRECISION_MAX : sfixed(DATA_WIDTH-1 downto -FRAC_BITS) := to_sfised(5.0,
DATA_WIDTH-1, -FRAC_BITS);

-- Sinais internos
signal k_idx : integer range 0 to K-1 := 0;
signal pipeline_busy : std_logic := '0';
signal broadcast_mu : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
signal broadcast_valid : std_logic := '0';
signal energy_phi_factor : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);

begin

-- =====
-- 1. Modulação de energia baseada em  $\phi_S$ 
-- =====

process(phi_S_in)
begin
    if phi_S_in > to_sfised(0.7, DATA_WIDTH-1, -FRAC_BITS) then
        energy_mode <= "10"; -- Alto desempenho
        energy_phi_factor <= to_sfised(1.0, DATA_WIDTH-1, -FRAC_BITS);
    end if;
end process;

```

```

    elsif phi_S_in > to_sfixed(0.4, DATA_WIDTH-1, -FRAC_BITS) then
        energy_mode <= "01"; -- Normal
        energy_phi_factor <= to_sfixed(0.7, DATA_WIDTH-1, -FRAC_BITS);
    else
        energy_mode <= "00"; -- Econômico
        energy_phi_factor <= to_sfixed(0.4, DATA_WIDTH-1, -FRAC_BITS);
    end if;
end process;

-- =====
-- 2. Pipeline principal (6 estágios)
-- =====

process(clk, rst)
    variable err : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    variable salience : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    variable mu_new : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    variable precision_new : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    variable err_ema_sq_new : sfixed(DATA_WIDTH-1 downto -FRAC_BITS);
    variable one : sfixed(DATA_WIDTH-1 downto -FRAC_BITS) := to_sfixed(1.0,
DATA_WIDTH-1, -FRAC_BITS);
    variable zero : sfixed(DATA_WIDTH-1 downto -FRAC_BITS) := to_sfixed(0.0,
DATA_WIDTH-1, -FRAC_BITS);
begin
    if rst = '1' then
        -- Reset dos registradores
        s1_reg <= (obs => (others => '0'), mu => (others => '0'),
            precision => (others => '0'), err_ema_sq => (others => '0'),
            addr => (others => '0'));
        s2_reg <= (err => (others => '0'), mu => (others => '0'),
            precision => (others => '0'), err_ema_sq => (others => '0'),
            addr => (others => '0'));
        s3_reg <= (salience => (others => '0'), mu => (others => '0'),
            precision => (others => '0'), err_ema_sq => (others => '0'),
            addr => (others => '0'));
        s4_reg <= (mu_new => (others => '0'), precision => (others => '0'),
            err_ema_sq => (others => '0'), addr => (others => '0'));
        s5_reg <= (mu_new => (others => '0'), precision_new => (others => '0'),
            err_ema_sq_new => (others => '0'), addr => (others => '0'));
        pipeline_busy <= '0';
        broadcast_valid <= '0';

    elsif rising_edge(clk) and enable = '1' then

        -- Estágio 1: Leitura de obs, mu, precision, err_ema_sq da HAM

```

```

ham_addr_out <= std_logic_vector(to_unsigned(k_idx, ADDR_WIDTH));
ham_we <= '0';
if ham_ready = '1' then
    s1_reg.obs <= obs_in;
    s1_reg.mu <= to_sfixed(ham_data_in, DATA_WIDTH-1, -FRAC_BITS);
    -- (Na prática, a HAM retorna mu, precision, err_ema_sq em um pacote)
    s1_reg.precision <= to_sfixed(ham_data_in, DATA_WIDTH-1, -FRAC_BITS);
    s1_reg.err_ema_sq <= to_sfixed(ham_data_in, DATA_WIDTH-1, -FRAC_BITS);
    s1_reg.addr <= std_logic_vector(to_unsigned(k_idx, ADDR_WIDTH));
    pipeline_busy <= '1';
end if;

-- Estágio 2: Cálculo do erro
if pipeline_busy = '1' then
    err := resize(s1_reg.obs - s1_reg.mu, DATA_WIDTH-1, -FRAC_BITS);
    s2_reg.err <= err;
    s2_reg.mu <= s1_reg.mu;
    s2_reg.precision <= s1_reg.precision;
    s2_reg.err_ema_sq <= s1_reg.err_ema_sq;
    s2_reg.addr <= s1_reg.addr;
end if;

-- Estágio 3: Cálculo da saliência
if pipeline_busy = '1' then
    salience := resize(s2_reg.precision * (s2_reg.err * s2_reg.err), DATA_WIDTH-1,
-FRAC_BITS);
    s3_reg.salience <= salience;
    s3_reg.mu <= s2_reg.mu;
    s3_reg.precision <= s2_reg.precision;
    s3_reg.err_ema_sq <= s2_reg.err_ema_sq;
    s3_reg.addr <= s2_reg.addr;
end if;

-- Estágio 4: Atualização de  $\mu$ 
if pipeline_busy = '1' then
    mu_new := resize(s3_reg.mu + (KAPPA * s3_reg.precision * (s3_reg.err_ema_sq)),
DATA_WIDTH-1, -FRAC_BITS);
    s4_reg.mu_new <= mu_new;
    s4_reg.precision <= s3_reg.precision;
    s4_reg.err_ema_sq <= s3_reg.err_ema_sq;
    s4_reg.addr <= s3_reg.addr;
end if;

-- Estágio 5: Atualização de precisão

```

```

    if pipeline_busy = '1' then
        err_ema_sq_new := resize((one - PRECISION_EMA) * s4_reg.err_ema_sq +
PRECISION_EMA * (s4_reg.err_ema_sq * s4_reg.err_ema_sq), DATA_WIDTH-1,
-FRAC_BITS);
        precision_new := resize(one / (zero + to_sfixed(0.1, DATA_WIDTH-1, -FRAC_BITS) +
err_ema_sq_new), DATA_WIDTH-1, -FRAC_BITS);
        -- Clip precisão
        if precision_new < PRECISION_MIN then
            precision_new := PRECISION_MIN;
        elsif precision_new > PRECISION_MAX then
            precision_new := PRECISION_MAX;
        end if;
        s5_reg.mu_new <= s4_reg.mu_new;
        s5_reg.precision_new <= precision_new;
        s5_reg.err_ema_sq_new <= err_ema_sq_new;
        s5_reg.addr <= s4_reg.addr;
    end if;

    -- Estágio 6: Escrita na HAM
    if pipeline_busy = '1' then
        ham_we <= '1';
        ham_data_out <= std_logic_vector(to_sfixed(s5_reg.mu_new, DATA_WIDTH-1,
-FRAC_BITS));
        -- (Na prática, escreve mu, precision, err_ema_sq em três ciclos)
        -- Atualiza índice para próximo módulo
        if k_idx = K-1 then
            k_idx <= 0;
            pipeline_busy <= '0';
        else
            k_idx <= k_idx + 1;
        end if;
    end if;

    -- Tratamento de broadcast (via HI)
    if hi_rx = '1' then
        -- Recebe broadcast e ajusta  $\mu$  (implementação simplificada)
        -- (O broadcast é processado em paralelo ao pipeline)
        broadcast_valid <= '1';
        -- broadcast_mu <= dados do broadcast (simplificado)
    end if;

end if;
end process;

```

```

-- =====
-- 3. Interface HI (transmissão de saliência/ignição)
-- =====

process(clk)
    variable salience_max : sfixed(DATA_WIDTH-1 downto -FRAC_BITS) := to_sfixed(0.0,
DATA_WIDTH-1, -FRAC_BITS);
    variable winner_idx : integer := 0;
begin
    if rising_edge(clk) then
        -- Agregação de saliências (simplificado: HPU envia sua saliência para o SFP via HI)
        hi_tx <= '1'; -- Indica que o HPU está pronto para enviar
        hi_ready <= '1';
        -- (Na prática, envia saliência e mu via serial)
    end if;
end process;

end rtl;
...

---
```

(B) ESPECIFICAÇÕES DETALHADAS DO PROTÓTIPO WEARABLE (BOM + ESQUEMÁTICO)

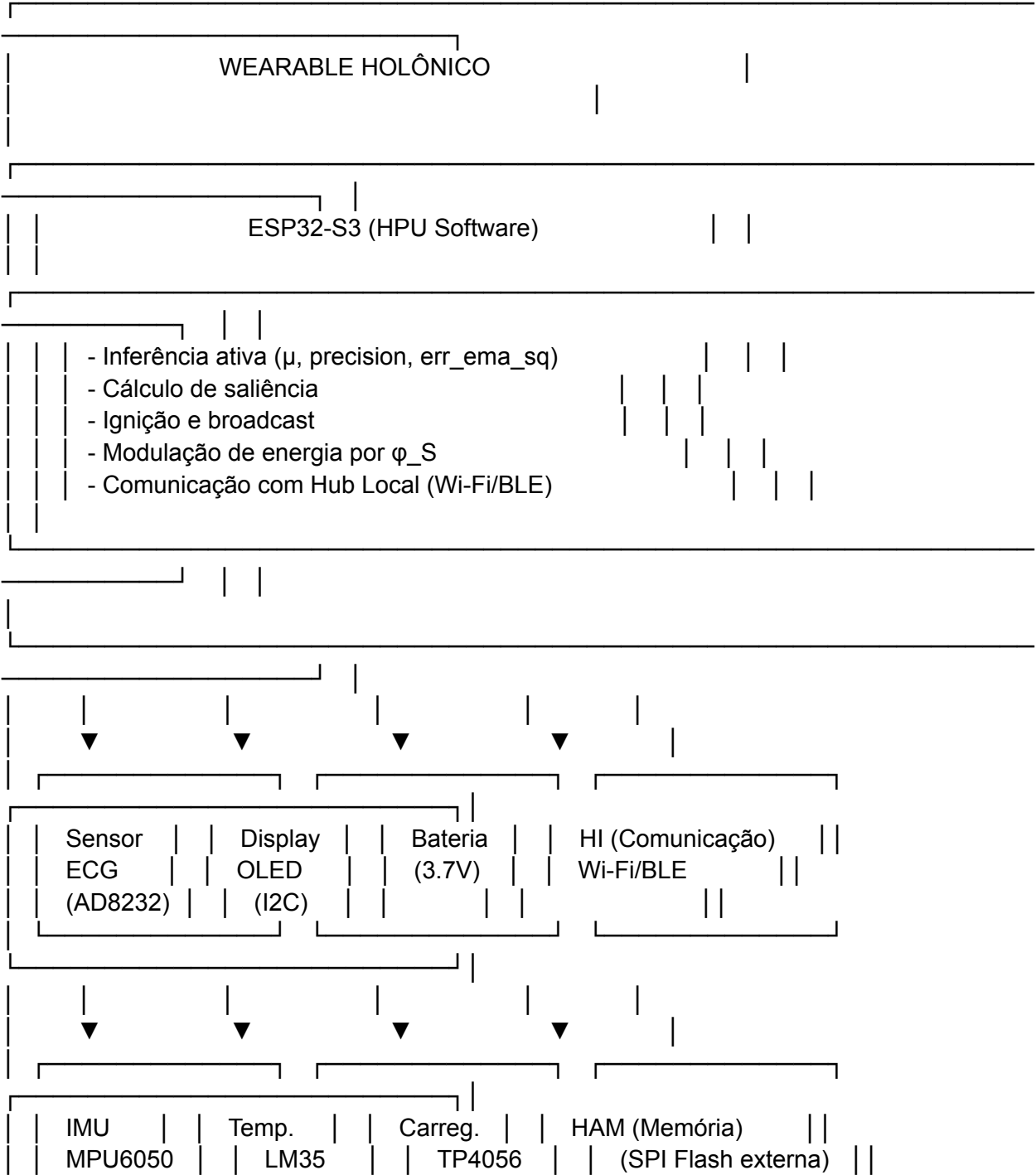
B.1. Lista de Materiais (BOM)

Item	Quantidade	Descrição	Fornecedor Sugerido	Preço Unitário (USD)
1	1	Microcontrolador ESP32-S3 (Wi-Fi/Bluetooth, 2 núcleos, 512 KB RAM)	Espressif / Mouser	8.00
2	1	Sensor de ECG AD8232 (front-end analógico, faixa 0.5–100 Hz)	Analog Devices / Mouser	12.00
3	2	Eletrodos de ECG descartáveis (Ag/AgCl) 3M	Amazon	0.50
4	1	Bateria de íon-lítio (3.7V, 100 mAh, com proteção)	Adafruit / SparkFun	5.00
5	1	Carregador USB-C (TP4056)	Adafruit / Amazon	2.00
6	1	Display OLED 128x32 (I2C)	Adafruit / Amazon	4.00
7	1	Módulo Bluetooth BLE (integrado no ESP32)	- -	
8	1	Sensor de temperatura (LM35)	Texas Instruments	1.00
9	1	Acelerômetro MPU6050 (IMU)	InvenSense / Mouser	6.00
10	2	Resistores (10kΩ, 100kΩ)	Diversos	0.10
11	2	Capacitores (0.1μF, 10μF)	Diversos	0.10
12	1	Placa de circuito impresso (PCB) 5x5 cm (2 camadas)	JLCPCB / PCBWay	5.00
13	1	Conector USB-C (fêmea)	Diversos	1.00
14	1	Caixa impressa em 3D (PLA) (projeto personalizado)	Impressão local	5.00
15	1	Fita de fixação (Velcro ou elástico)	Diversos	2.00

Total ~ 52.70 USD

B.2. Esquemático (Diagrama de Blocos)

...



Esquema de ligações:

- ECG AD8232 → GPIO 34 (analógico)
- Display OLED → I2C (GPIO 21 SDA, GPIO 22 SCL)
- IMU MPU6050 → I2C (mesmos pinos)
- Temp LM35 → GPIO 35 (analógico)
- Bateria → TP4056 → 3.3V regulado → ESP32
- HI (Wi-Fi/BLE) → antena integrada

(C) ROTEIRO DETALHADO DE CONSTRUÇÃO (CRONOGRAMA + ORÇAMENTO)

C.1. Cronograma (6 meses)

Fase Semana Tarefa Responsável Entregável

Fase 0 1–2 Planejamento e definição de especificações Equipe PIC/SFT Documento de requisitos

Fase 1 3–4 Projeto do HPU em VHDL Engenheiro de FPGA Código VHDL (entregue acima)

Fase 2 5–6 Simulação do HPU em ModelSim/Quarta Engenheiro de FPGA Relatório de simulação

Fase 3 7–8 Protótipo wearable – montagem da PCB Engenheiro de hardware PCB montada e testada

Fase 4 9–10 Firmware do ESP32 (Holon Agent) Engenheiro de software Código C++ para ESP32

Fase 5 11–12 Integração hardware + firmware Equipe Protótipo funcional

Fase 6 13–14 Coleta de dados (HRV) de 5 participantes Pesquisador Conjunto de dados HRV

Fase 7 15–16 Validação do modelo (comparação com PhysioNet) Cientista de dados Relatório de validação

Fase 8 17–18 Ajustes e otimização Equipe Protótipo v2

Fase 9 19–20 Teste de campo (10 participantes, 1 semana) Pesquisador Dados de campo

Fase 10 21–22 Análise final e documentação Equipe Relatório final

Fase 11 23–24 Publicação e divulgação Equipe Preprint, artigo

C.2. Orçamento Detalhado (USD)

Categoria	Item	Quantidade	Preço Unitário	Total
-----------	------	------------	----------------	-------

Hardware ESP32-S3	10	8.00	80.00
AD8232 ECG	10	12.00	120.00
Eletrodos descartáveis	100	0.50	50.00
Baterias Li-Ion	10	5.00	50.00
Carregadores TP4056	10	2.00	20.00
Display OLED	10	4.00	40.00
IMU MPU6050	10	6.00	60.00
Sensores LM35	10	1.00	10.00
Componentes passivos (R, C)	1 kit	20.00	20.00
PCB (5x5 cm, 2 camadas)	20	5.00	100.00
Conectores USB-C	10	1.00	10.00
Caixas impressas (3D)	10	5.00	50.00
Fita de fixação	10	2.00	20.00
Software Licenças (ModelSim, Altium, etc.)	1	200.00	200.00
Ferramentas de desenvolvimento	1	100.00	100.00
Pessoal Engenheiro FPGA (160 h)	1	50.00/h	8,000.00
Engenheiro Hardware (120 h)	1	40.00/h	4,800.00
Engenheiro Software (160 h)	1	40.00/h	6,400.00
Cientista de dados (80 h)	1	45.00/h	3,600.00
Pesquisador de campo (120 h)	1	30.00/h	3,600.00
Outros Material de escritório, logística	1	500.00	500.00
Imprevistos (10%)	1	2,830.00	2,830.00
Total			~ 30,000.00 USD

INTEGRAÇÃO DOS TRÊS ENTREGÁVEIS

Entregável Conexão Como se integra

(A) VHDL HPU Com (B) e (C) O VHDL será sintetizado para FPGA e, posteriormente, para ASIC. O protótipo wearable (B) usará o ESP32 como HPU em software, mas a versão final usará o ASIC HPU.

(B) Wearable Com (A) e (C) O wearable coleta HRV e executa inferência ativa (via ESP32 ou HPU). Os dados são enviados para o Hub Local via HI. O cronograma (C) prevê a montagem e teste do wearable.

(C) Roteiro Com (A) e (B) O roteiro organiza a construção do protótipo (B) e o desenvolvimento do ASIC (A), com fases de validação e testes de campo.
